

Министерство Образования и исследований

Государственный Университет Молдовы

Кафедра информатики

Отчет:

По дисциплине Java Script

Лабораторная работа No5

Тема: Работа с DOM-деревом и событиями в JavaScript

Выполнил:

Чубару Максим, gr. IA2503

Проверил:

N.Calin

Кишинев, 2026

Цель работы

Ознакомиться с основами взаимодействия JavaScript с DOM-деревом на примере веб-приложения для учета личных финансов, изучить работу с HTML-элементами, событиями, таблицами, формами и модулями JavaScript.

Ход работы

1. Настройка структуры проекта

Была создана структура проекта:

```
project/
├── index.html
├── style.css
└── src/
    ├── index.js
    ├── transactions.js
    ├── ui.js
    └── utils.js
```

Файл `index.html` подключался к `index.js` с использованием:

```
<script type="module" src="./src/index.js"></script>
```

Это позволило использовать ES6-модули и разделить код по функциональности.

Для хранения данных использовался массив `transactions`, содержащий объекты транзакций. При добавлении новой записи функция `addTransaction()` получала данные из формы, выполняла проверку корректности введенной информации и создавала объект транзакции:

```
const transaction = {
  id: generateID(),
  date: new Date(),
  amount,
  category,
  description
};
```

После создания объект добавлялся в массив:

```
transactions.push(transaction);
```

Для отображения транзакций использовалась функция `renderRow()`, которая динамически создавала строку таблицы через DOM:

```
const row = document.createElement("tr");
```

В зависимости от суммы транзакции строка окрашивалась в зеленый или красный цвет:

```
if (transaction.amount > 0) {
  row.style.backgroundColor = "lightgreen";
} else {
  row.style.backgroundColor = "lightcoral";
}
```

В таблице отображалось краткое описание транзакции — первые четыре слова текста:

```
const shortDescription =
  transaction.description.split(" ").slice(0, 4).join(" ");
```

Для удаления транзакций применялось делегирование событий. Обработчик клика устанавливался на элемент `<table>`, после чего определялось, была ли нажата кнопка удаления:

```
table.addEventListener("click", function(event) {

  if (event.target.tagName === "BUTTON") {

    const id = event.target.dataset.id;

  }

});
```

После получения идентификатора выполнялся поиск нужной транзакции через `findIndex()`, удаление элемента из массива методом `splice()` и удаление строки таблицы через `remove()`.

Для подсчета общей суммы транзакций была реализована функция

```
calculateTotal(), использующая метод reduce():

const total = transactions.reduce(
  (sum, transaction) => sum + transaction.amount,
  0
);
```

Результат автоматически отображался на странице в отдельном HTML-элементе.

Также была реализована возможность просмотра полного описания транзакции. При нажатии на строку таблицы выполнялся поиск соответствующего объекта и отображение полного текста описания:

```
document.querySelector("#full-description").textContent =
  transaction.description;
```

В результате была создана интерактивная HTML-страница, позволяющая добавлять, удалять и просматривать транзакции, а также автоматически подсчитывать общий баланс пользователя.

Ссылка на репозиторий Git: <https://github.com/maximcv246-a11y/Lab5-JS>

Внешний вид сайта:

введите транзакцию:

сумма
категория
краткое описание
отправить

Дата и время	Категория транзакции	Краткое описание транзакции	Действие
1/1/2026, 2:00:00 AM	bank	paying loan for a	Удалить
1/2/2026, 2:00:00 AM	job	got my salary	Удалить
1/3/2026, 2:00:00 AM	bank	taxes for real estate	Удалить
1/4/2026, 2:00:00 AM	job	best worker monthly bonus	Удалить
1/5/2026, 2:00:00 AM	supermarket	groceries for a week	Удалить

Общая сумма: 941000

Полное описание:

Нажмите на транзакцию

Вывод

В ходе выполнения лабораторной работы были изучены основные возможности JavaScript для взаимодействия с DOM-деревом и создания динамических веб-страниц. В процессе работы было разработано веб-приложение для учета личных финансов, позволяющее добавлять, отображать и удалять транзакции, а также автоматически подсчитывать общую сумму операций. Были освоены методы работы с HTML-элементами, обработчиками событий, формами и таблицами, изучены принципы делегирования событий и динамического изменения содержимого страницы. Также были получены практические навыки использования модульной структуры проекта, работы с массивами объектов и встроенными методами JavaScript для обработки данных. Полученные знания и навыки могут применяться при разработке современных веб-приложений, пользовательских интерфейсов, административных систем, финансовых сервисов и других интерактивных сайтов.

Ответы на контрольные вопросы

1. Каким образом можно получить доступ к элементу на веб-странице с помощью JavaScript?

Для получения доступа к элементам DOM используются методы:

```
document.querySelector()  
document.querySelectorAll()  
document.getElementById()  
document.getElementsByClassName()
```

Пример: `const element = document.querySelector("#title");`

2. Что такое делегирование событий и как оно используется?

Делегирование событий — это способ обработки событий через родительский элемент вместо установки обработчика на каждый дочерний элемент.

Пример:

```
table.addEventListener("click", function(event) {  
  
    if (event.target.tagName === "BUTTON") {  
        console.log("Нажата кнопка");  
    }  
  
});
```

Преимущества:

- меньше обработчиков;
 - выше производительность;
 - работает для динамически созданных элементов.
-

3. Как можно изменить содержимое элемента DOM после его выборки?

Содержимое можно изменить через:

```
element.textContent  
element.innerHTML
```

Пример:

```
document.querySelector("#title").textContent = "Новый текст";
```

4. Как можно добавить новый элемент в DOM дерево с помощью JavaScript?

Для добавления элемента используются:

```
document.createElement()  
appendChild()
```